

Content-Addressable Accountability: A Line-Hash Approach to Code Ownership

The OWNH Framework

Objective Weighted Normalized Heuristics

<https://ownh.org>

Abstract

Engineering organizations need to know who owns their code. Existing answers depend on activity (commit counts), recency (`git blame`), or declarations (`CODEOWNERS` files), and each can be changed by the behavior it is meant to measure. This paper presents OWNH, a method that assigns every line of code to exactly one owner using only the line's content hash and the version history of the repositories that contain it. The owner of a line is the first contributor, across all repositories under analysis, to commit a line with that exact content. The assignment is deterministic, independent of the order in which repositories are analyzed, and permanent for a fixed scope of analyzed histories. The definitions, the attribution procedure, the benchmark against established methods, and the derived governance metrics: ownership leverage, code demolition, code survival, and quarterly projections suitable for objectives and key results, are described.

1 Introduction

Ownership measures have a respectable research history. In Windows Vista and Windows 7, ownership measures correlated with pre-release faults and post-release failures [1]. Research comparing ownership definitions finds that line-based ownership suits accountability and attribution, that commit-based ownership is the better choice for bug fixing and quality planning, and that commit-based and line-based methods disagree about who the major developers are [2]. The question is therefore not whether ownership matters, but which definition an organization can defend. The method presented here addresses accountability.

Each conventional definition carries a human element:

- **Commit counts** reward volume. Ten small commits outrank one good one.
- `git blame` reports the revision and author that last modified each line [3]. Recency is not authorship.
- `CODEOWNERS` files are declarations. They are maintained by people and are out of date by the time they merge.

In the method presented here, the human element is removed. The history is read once, and a single answer is produced for every line, which any party can regenerate from the same repositories.

2 Definitions

Definition (Line). A line is the exact sequence of bytes between two newline characters. No normalization is applied. Leading and trailing whitespace, indentation, blank lines, and a

trailing carriage return are part of the line. A line ending in `\r\n` and the same line ending in `\n` are different lines.

Definition (Binary file). A file that git classifies as binary, using its attributes and content check, is treated as a single line whose content is the entire file. Git’s classification is used as is; no other heuristic is applied.

Definition (Line hash). The identity of a line is $h = \text{SHA-256}(\ell)$, the hash of its bytes. Two lines are the same line if and only if their hashes are equal, regardless of file, path, language, or repository.

Line (shown between brackets)	SHA-256 prefix	Remark
[]	e3b0c44298fc1c14	a blank line
[}]	d10b36aa74a59bcf	closing brace
[]}]	737db166c79ae98e	closing brace, one level in
[}]	28d86778615f6af4	closing brace, two levels in
[\r]	9d1e0e2d9459d065	a blank line, CRLF

Table 1: Line hashes. Each row is a distinct line with its own owner.

Definition (Introduction). A commit c introduces hash h if the diff of c against its parent adds a line with hash h . Each introduction carries the key

$$k(c) = (t_{\text{author}}(c), \text{repository}(c), \text{position}(c)),$$

where t_{author} is the author time, repository is the name of the repository, unique within an analysis, and position is the commit’s index in the canonical traversal of that repository’s history: git’s topological order (`-topo-order`), reversed so that the oldest commit comes first. A history admits several topological orders; git computes this one deterministically for a given history.

Definition (Owner). The owner of hash h is the author of the introduction with the smallest key:

$$\text{owner}(h) = \text{author}\left(\arg \min_{c \in I(h)} k(c)\right),$$

where $I(h)$ is the set of commits, in any repository under analysis, that introduce h . Every occurrence of h , in every file of every repository, has this owner.

Definition (Identity). Contributors are identified by email address, compared without regard to case, after applying each repository’s `.mailmap`. The display name of an identity is the name it has used most often.

3 Attribution

3.1 Introductions and removals

The complete history of every repository is considered. Each line a commit adds relative to its parent is an introduction of that line’s hash, and the owner of each hash follows from the smallest key among its introductions. Each line a commit removes is a removal, recorded by hash, quarter, and the identity of the commit’s author, for the analyses in Section 6.

3.2 Merge commits

A merge commit is compared with the result of merging its parents automatically. The lines it adds relative to that result are the lines its author resolved by hand, and they are introductions

like any other. Merge commits are not counted as commits, and lines they remove are not counted as removals.

3.3 Lines at head

Every line of every file at the head of each repository is attributed to the owner of its hash. The total is the number of *lines under management*. Each line counts once per occurrence: a hash that appears in many places contributes all of those places to its owner.

4 Properties

Property (Uniqueness). Every line has exactly one owner. Ties are impossible: two introductions with equal author time are separated by repository name, and within a repository by position in the canonical traversal.

Property (Determinism). The same repositories at the same commits produce the same owners and the same counts.

Property (Order independence). Ownership is a minimum over a key that does not depend on the order of analysis. The result does not depend on the order in which repositories are analyzed.

Property (Permanence). An owner keeps ownership regardless of later activity, inactivity, or employment status. Ownership is permanent for a fixed scope: for a given set of analyzed histories, ownership of a line changes only when the line changes. Extending the scope with a repository that introduces the same line earlier, or rewriting an analyzed history, transfers ownership to the earlier introduction.

Property (Global scope). Ownership is shared across repositories. A line first written in one repository is owned by its first author in every repository where it occurs.

4.1 Operational consequences

The following follow directly from the definitions and are reported as designed:

- **Formatting.** A formatting commit changes the hash of every line it reformats. Lines whose new form has not been written before are introduced by the author of the formatting commit, who becomes their owner.
- **Renames.** Renaming an identifier changes every line that contains it, with the same effect.
- **Reverts.** A revert restores earlier lines. Their hashes already have owners, so ownership returns to the original authors.
- **Indentation.** Each indentation level of a closing brace is a separate line with its own owner.
- **Copying.** A copied line keeps the owner it already has, wherever it is pasted.
- **Blank lines.** The blank line is one line. It has one owner.

5 Benchmarking against established methods

For each repository, and for all repositories combined, the top owner is determined under three definitions:

1. **Commits**: the identity with the most non-merge commits.
2. **Blame**: the identity with the most lines at head according to `git blame`.
3. **Line hash**: the identity with the most lines at head by first introduction.

For each repository, agreement is recorded when the three definitions name the same owner.

6 Governance metrics

6.1 Ownership leverage

Leverage is the number of lines an identity owns per line it wrote, where lines written are the lines added in that identity's own commits:

$$\text{leverage}(o) = \frac{\text{lines owned at head}}{\text{lines added in own commits}}.$$

A leverage above one means an identity's lines have been reproduced, by others or by itself, more often than they were written.

6.2 Code demolition

A removal is attributed to the author of the removing commit and counted against the owner of the removed line. From the recorded removals are derived: the lines each identity removed that were owned by someone else, the lines each owner lost to others, and the lines added again after they had first been removed. A restored line returns to its original owner.

6.3 Code survival

Each added line is followed from the quarter it was added. Copies of the same hash cannot be told apart, so removals are paired with the oldest surviving copies first. A paired line dies at its age in quarters; a line still present is censored at its current age. Survival is the Kaplan–Meier estimate over these ages, overall and per owner, in the tradition of cohort analyses of surviving code [7]. The *code half-life* is the first age at which the estimate falls to one half or below. When the estimate never falls that far, the half-life is extrapolated with an exponential through the last point and labeled as projected.

6.4 Outlook

The ownership series is the cumulative net of recorded additions and removals per owner and quarter, summed over all branches of the history. It is not a sequence of snapshots: removals inside merge commits are not counted, so lines added on a branch whose changes a merge later discarded remain in the series. The difference from the lines at head is stated with each outlook. “Now” is the quarter of the newest commit. An identity is *inactive* in a quarter if it has no commit in that quarter or the three before it. Three series are projected:

- the share of lines owned by the principal owner;
- the share of lines owned by inactive identities;
- the number of blank lines.

Each projection is a least-squares line fitted over the last 12 quarters, with at least three quarters required, continued from the latest value and reported with its coefficient of determination R^2 .

For a share, the projected result is the quarter in which it reaches a majority, or the quarter since which it has held one. For blank lines, it is the quarter in which the next milestone is reached, where milestones are the round numbers 1, 2, 5, 10, 20, 50, ... A result beyond 40 quarters is stated as not expected within that horizon.

7 Related work

Studies built on World of Code identify the repository that first committed each version of a file as its origin [4]. The Software Heritage provenance service answers where a file comes from with the oldest revision that contains it [5]. Here the same first-introducer principle is applied at the granularity of a single line. Truck-factor research derives key developers from version history [6]. `git blame` offers options to follow moved and copied lines and to ignore formatting revisions [3]; in this method every revision is credited as written.

8 Conclusion

Under OWNH, the question of code ownership is settled by a definition that requires no judgment: the first contributor to write a line owns it, everywhere and permanently. Every line has an owner. Every owner can be named. Someone has to own it.

References

- [1] C. Bird, N. Nagappan, B. Murphy, H. Gall, P. Devanbu. *Don't Touch My Code! Examining the Effects of Ownership on Software Quality*. ESEC/FSE 2011. <https://www.microsoft.com/en-us/research/publication/dont-touch-my-code-examining-the-effects-of-ownership-on-software-quality/>
- [2] *Code Ownership: The Principles, Differences, and Their Associations with Software Quality*. arXiv:2408.12807. <https://arxiv.org/abs/2408.12807>
- [3] *git-blame documentation*. <https://git-scm.com/docs/git-blame>
- [4] M. Jahanshahi, D. Reid, A. Mockus. *Beyond Dependencies: Copy-Based Reuse*. arXiv:2409.04830. <https://arxiv.org/abs/2409.04830>
- [5] Software Heritage. *Provenance service documentation*. <https://docs.softwareheritage.org/devel/swl-provenance/>
- [6] G. Avelino, M. T. Valente, et al. *What is the Truck Factor of popular GitHub applications?* <https://peerj.com/preprints/1233>
- [7] *git-of-theseus*. <https://github.com/erikbern/git-of-theseus>